

StudyAI: AI-Powered Study Companion & Quiz Generator

Project Description

StudyAI is a web-based AI-powered study companion and quiz generation platform designed to help students transform their notes, textbooks, and study materials into structured learning content in one place. The application was developed as a practical college project with a focus on personalized learning, exam preparation, and intelligent scheduling. It provides a single dashboard where students can upload study material, generate concise summaries, create flashcards, take AI-generated quizzes, and receive adaptive study schedules.

The system is built using React.js for the frontend, Python Flask as the backend service, and Firebase as the cloud data store. The application integrates the Groq API using the Llama 3.3 70B Versatile model for all AI-powered features including summarization, flashcard generation, quiz creation, adaptive recommendations, and personalized study plan generation.

The project evolved into a fully functional AI study platform that supports appointment management, performance analytics, weak topic identification, and exportable study schedules. It was deployed with a React development server for the frontend and a Flask backend with local JSON fallback storage when Firebase credentials are not configured.

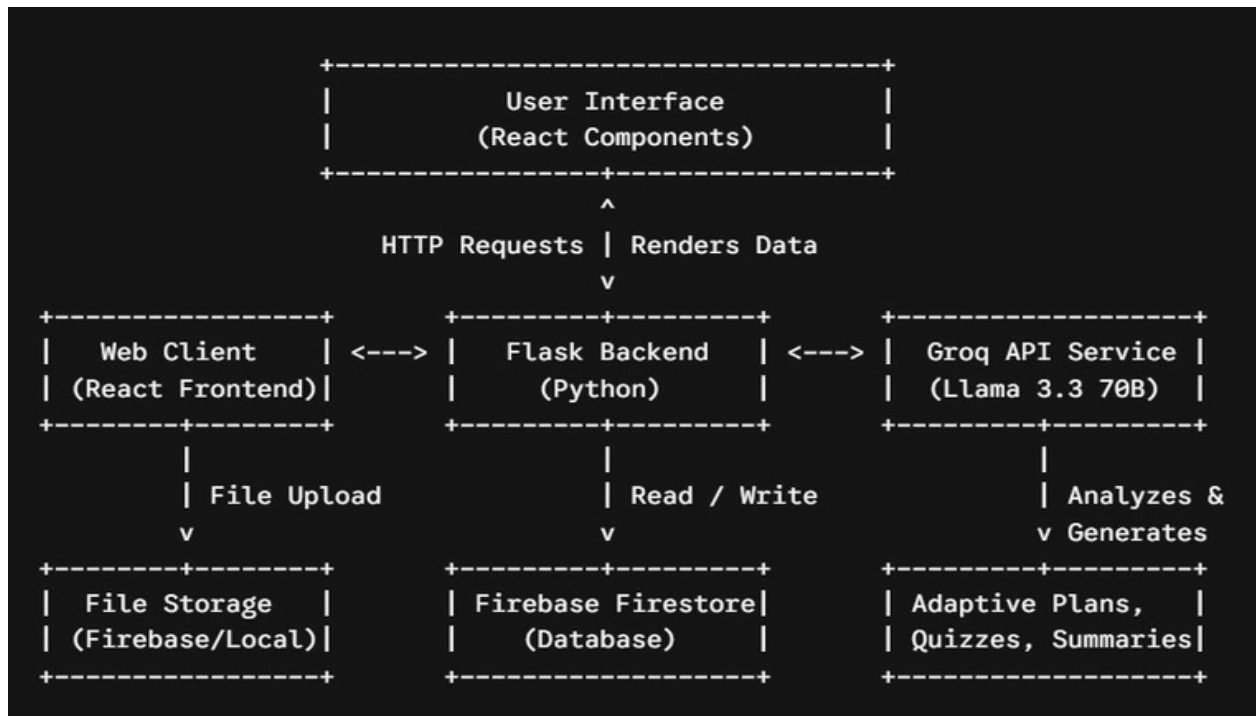
Scenarios

- **Scenario 1:** A student uploads a PDF chapter, DOCX file, or pastes raw text into the upload interface. The system extracts the content, stores the material with a unique ID, and makes it available for all AI-powered processing features including summary generation, flashcard creation, and quiz generation.

- **Scenario 2:** A student generates a summary from their uploaded material. The Groq-powered AI reads the full content and returns a structured markdown summary with a topic overview, key concepts, important definitions, core principles, and bullet-point revision notes for last-minute review.
Scenario 3: A student creates a set of flashcards from their study material.
- The AI generates question-and-answer pairs with topic labels and difficulty levels. The student flips through cards, marks known cards, shuffles the deck, and tracks how many concepts they have mastered.
Scenario 4: A student selects a quiz type (MCQ, True/False, or Short Answer), chooses the number of questions, and takes an AI-generated quiz. After submission, the system calculates the score, displays correct answers with explanations, identifies weak topics, and saves performance data to the dashboard.
- **Scenario 5:** The student opens the Schedule page to generate a personalized 7-day study plan. The AI uses the uploaded material content and any weak topics identified from quiz results to create a day-by-day schedule with tasks, time slots, focus topics, and study tips.

Technical Architecture

As shown in image below StudyAI uses a robust web architecture:



Architecture Description

StudyAI utilizes a full-stack detached client-server architecture to provide a fast and highly interactive AI learning experience:

- **Frontend:** Built completely with **React.js**, creating a responsive SPA (Single Page Application) dashboard mapping analytics, interactive quiz interfaces, dynamic flashcard flipping, and custom file uploads.
- **Backend:** Built on **Python Flask**, which serves as a REST API layer managing client requests, orchestrating file parsing, structuring prompt payloads for safe execution, and serving response fallbacks.
- **Database & Storage:** Uses **Firebase Firestore** for storing semi-structured metadata like user quiz history, analytics variables, text extracts, and structured schedules. File binaries are handled via **Firebase Storage**.

Software Requirements

- **Operating System:** Windows 10 / 11, macOS (Monterey or later), or Linux (Ubuntu 20.04+).

- **Web Browser:** Latest version of Google Chrome, Mozilla Firefox, or Microsoft Edge.
- **Core Libraries & Frameworks:** React.js, Python Flask, Firebase SDK, Axios, React Markdown, Chart.js.

AI Core Integration: Interfaces seamlessly with the **Groq API Service** running **Llama 3.3 70B Versatile** to run complex prompt engineering setups for instant markdown summary returns and deterministic evaluation structures.

Pre-requisites & System Requirements

Hardware Requirements (Refer to image_39f758.png)

- **Processor:** Intel Core i5 (10th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** Minimum 8GB (Recommended: 16GB for smooth AI model execution, browser multitasking, and cloud-based development).
- **Storage:** 256GB SSD minimum (Recommended: 512 GB SSD for faster builds and project storage).
- **GPU (Optional):** NVIDIA GTX 1650 or above for AI/ML experimentation and local model testing.
- **Internet Connectivity:** Stable high-speed internet connection (minimum 10 Mbps recommended for API usage, GitHub deployment, cloud IDEs, and AI integrations).

Project Workflow & Milestones

Milestone 1: Requirement Analysis and Architecture

- **Activity 1.1:** Generate and Configure the Groq AI Service API Key.
- **Activity 1.2:** Configure Required Accounts and Services (Firebase setup).
- **Activity 1.3:** Research and Select the Appropriate Generative AI Model (Llama 3.3 70B Versatile).
- **Activity 1.4:** Define the Architecture of the Application (Draft detailed sequence schemas).

- **Activity 1.5:** Set Up the Development Environment (React scaffold and Flask virtualenv structure).

Milestone 2: Core Personalities & Feature Development

- **Activity 2.1:** Develop Core AI Study Companion Features (Parsing templates and structured JSON prompt contexts).
- **Activity 2.2:** Implement the Python Flask Backend (Create request controllers and error handling endpoints).

Milestone 3: Backend Logic Implementation

- **Activity 3.1:** Writing the Main Application Logic (`app.py` workflow configuration, data routers, API state connections).

Milestone 4: Frontend & Interface Development

- **Activity 4.1:** Designing and Developing the User Interface (Responsive components on React dashboard).
- **Activity 4.2:** Creating Dynamic Content Rendering with React (State handlers for quizzes, charts, and dynamic content displays).

Milestone 5: Deployment & Integration Testing

- **Activity 5.1:** Preparing the Application for Local Deployment.
- **Activity 5.2:** Deploy the Frontend and Backend Services locally or via test cloud providers.
- **Activity 5.3:** Validate Deployment and Test Live Workflow.

Step-by-Step Implementation Guide

Step 1: Environment Setup & API Configuration

Before starting, initialize your virtual dependencies and securely lock your development environment configurations:

1. Create a `.env` file in the backend root directory

Code snippet

```
GROQ_API_KEY=your_secure_groq_api_key_here
FIREBASE_API_KEY=your_firebase_key_here
FLASK_ENV=development
```

2. Initialize and load inside your backend server (`app.py` or configuration setup):

Python

```
from dotenv import load_dotenv
import os
from groq import Groq

load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

3. Create your backend virtual environment to run cleanly:

Bash

```
python -m venv myenv
# Activate on Windows:
myenv\Scripts\activate
pip install -r requirements.txt
```

Step 2: Backend Main Routing Configuration

Set up your core operational endpoints to safely pipe React client requests straight into the execution layer:

```
@app.route("/api/generate-summary", methods=["POST"])
def generate_summary():
    data = request.get_json()
    material_content = data.get("content", "").strip()

    if not material_content:
        return jsonify({"error": "Content field cannot be empty"}), 400

    try:
        chat_completion = client.chat.completions.create(
            messages=[
                {
                    "role": "system",
                    "content": "You are an advanced academic companion. Return summaries strictly formatted in markdown."
                },
                {
                    "role": "user",
                    "content": f"Summarize the following material:\n\n{material_content}"
                }
            ],
            model="llama-3.3-70b-versatile",
            temperature=0.5,
            max_tokens=2048
        )
        response_text = chat_completion.choices[0].message.content
        return jsonify({"success": True, "summary": response_text})
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Website User Interface Sections

1. Dashboard & Upload Hub

- **Description:** Provides an all-in-one centralized view with dark-glass thematic card containers. Features an interactive cloud upload node supporting drag-and-drop actions for PDF/TXT files, quick-paste field areas, and real-time status monitors for raw document vector extraction.

2. AI Quiz Deck & Interaction Field

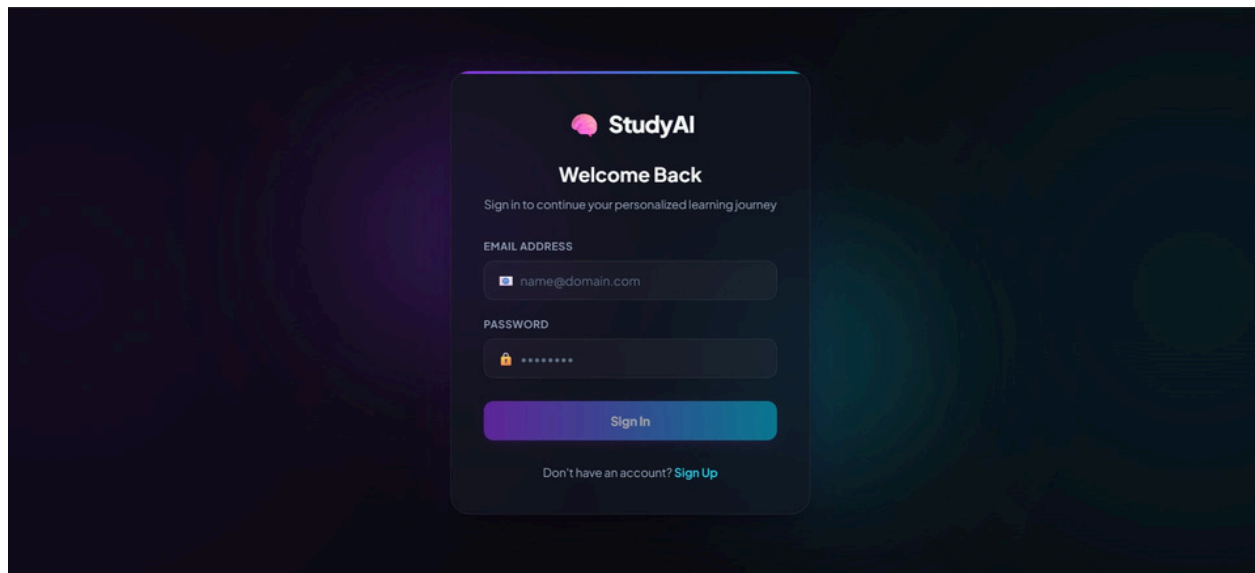
- **Description:** Renders real-time question sheets tracking choices dynamically via active hooks. Once answered, it alters card borders to clear green/red states to display validity, unlocks collapsible explanation paragraphs, and shows dynamic percentage rings on completion panels.

3. Analytics Progress Boards

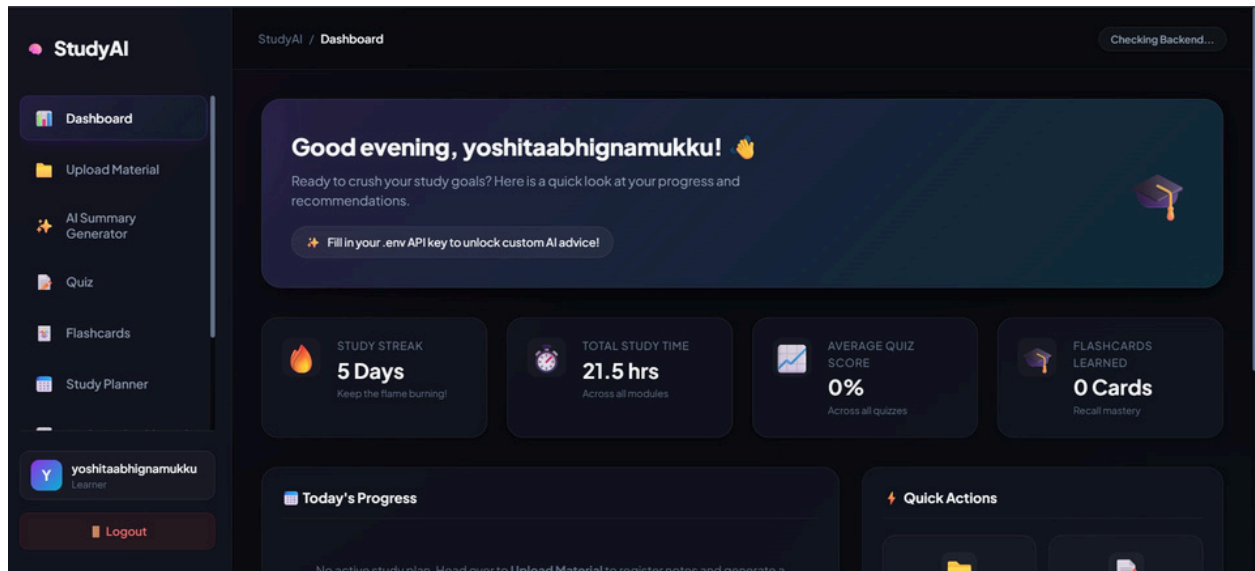
- **Description:** Integrates dynamic JavaScript tracking dashboards (via Chart.js or responsive graphs) processing performance metrics over time. It highlights localized danger labels noting "Weak Topics" directly based on AI evaluation scoring histories.

Exploring the Website's Web Pages:

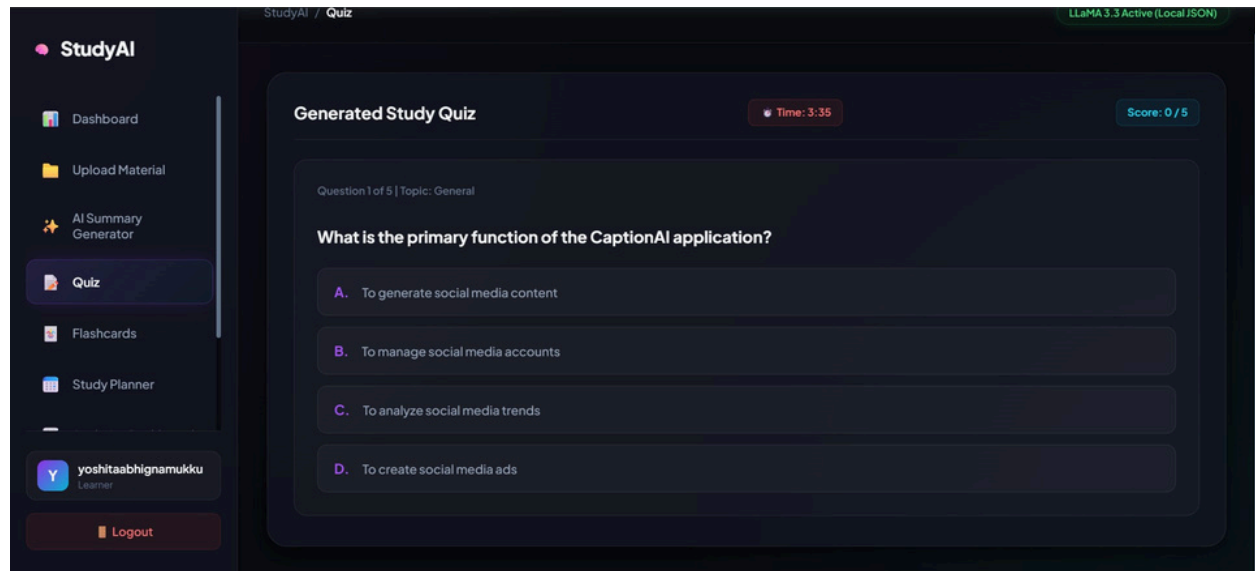
Home page:



Input Section:



Results Section:



Conclusion

StudyAI leverages Generative AI technology to deliver an efficient, structured solution for modern student learning constraints. By routing optimized engineering contexts to Groq's Llama 3.3 70B model, it delivers instantaneous learning maps, summaries, and adaptive plans without traditional structural overhead. The flexible, decoupled stack architecture paves the way for direct scale upgrades, including voice interaction models, localized team collaboration spaces, and fully direct vector-space file search lookups.

Project Done by:

Mukku Yoshita Abhigna

Rangavallika Sajja

Sai Priyanka Thammineni

Buchiboiena Devi sri